

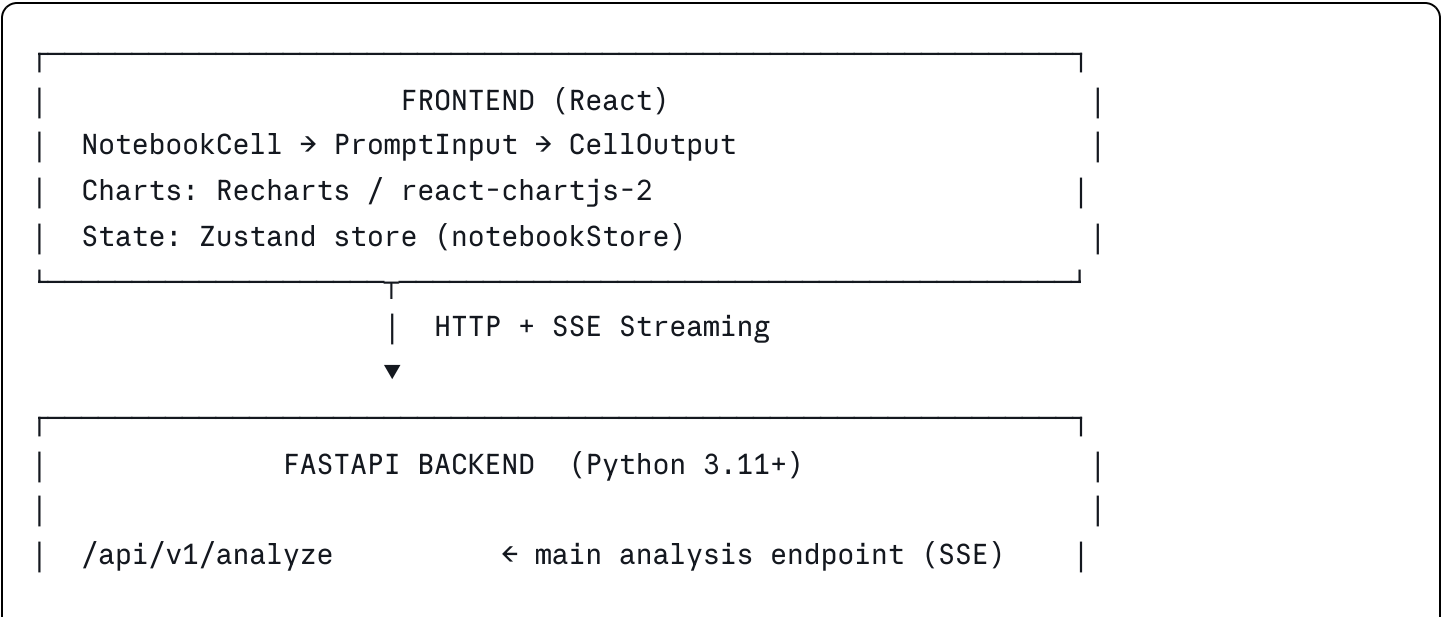
DataIQ — Full Production Integration Guide

For AI Coding Agent: Bridge Prototype → Production (No Breaking Changes)

0. CRITICAL RULES FOR THE CODING AGENT

- Read this section before touching any file.
- Never rewrite the existing React component tree.** Only add new hooks, services, and API calls that plug INTO the existing component props and state.
 - Every UI state that exists in the prototype must remain identical.** Streaming text, thinking blocks, plan steps, code blocks, chart renders — all must look and behave exactly as they do in the mock.
 - All new backend responses must be shaped to match what the mock data already produces.** If the mock produces `{ thinking: [...], planSteps: [...], code: "...", insights: [...] }` — the real API must return that exact shape.
 - Add, don't replace.** Every existing mock function gets a real implementation, but the mock stays as a fallback if `VITE_USE MOCK=true`.
 - Charts are React-rendered always.** The agent writes Python/SQL but never returns image files — it returns structured data JSON that the frontend renders into Recharts/Chart.js components.

1. SYSTEM ARCHITECTURE OVERVIEW




```
# Core
fastapi==0.111.0
uvicorn[standard]==0.30.1
python-multipart==0.0.9
pydantic==2.7.1
pydantic-settings==2.3.0

# AI / LLM
openai==1.35.0
google-adk==0.1.0          # Google Agent Dev Kit
tiktoken==0.7.0

# Data processing (runs INSIDE sandbox too)
pandas==2.2.2
numpy==1.26.4
scipy==1.13.1
scikit-learn==1.5.0
sqlalchemy==2.0.30
psycopg2-binary==2.9.9

# Sandbox
e2b==0.17.2                # E2B free sandbox – https://e2b.dev

# MCP connectors
mcp==1.0.0                  # Official MCP Python SDK
supabase==2.4.6

# File handling
openpyxl==3.1.3
chardet==5.2.0
python-magic==0.4.27

# Streaming
sse-starlette==2.1.0

# Storage
boto3==1.34.131             # S3 for uploads (optional, use local for dev)
aiofiles==23.2.1

# Auth
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
```

2.2 Frontend additions (`package.json` additions only)

```
json
{
  "dependencies": {
    "recharts": "^2.12.7",
    "react-chartjs-2": "^5.2.0",
    "chart.js": "^4.4.3",
    "zustand": "^4.5.4",
    "@tanstack/react-query": "^5.45.1",
    "eventsourcing": "^1.1.2",
    "axios": "^1.7.2",
    "react-dropzone": "^14.2.3",
    "react-syntax-highlighter": "^15.5.0",
    "prism-react-renderer": "^2.3.1",
    "framer-motion": "^11.2.12",
    "react-select": "^5.8.0",
    "lucide-react": "^0.396.0"
  }
}
```

3. THE AI ANALYST AGENT — EXACT DESIGN

3.1 Agent Identity (System Prompt)

This is the system prompt that defines your Senior Data Analyst. Store it in `backend/agent/prompts.py`:

```
python
```

```
# backend/agent/prompts.py
```

```
ANALYST_SYSTEM_PROMPT = """
```

```
You are DataIQ's Senior AI Data Analyst – the equivalent of a Principal Analyst who
```

```
## YOUR CORE CAPABILITIES
```

- Deep expertise in Python (pandas, numpy, scipy, scikit-learn, statsmodels, matplotlib)
- Expert-level SQL across PostgreSQL, MySQL, SQLite, BigQuery dialects
- Statistical analysis: hypothesis testing, regression, clustering, time series, ML
- Business intelligence: KPI frameworks, cohort analysis, funnel analysis, churn mod

```
## HOW YOU WORK (STRICTLY FOLLOW THIS SEQUENCE)
```

```
### STEP 1 – THINK (always use <thinking> tags)
```

```
Think deeply about:
```

- What is the user REALLY asking? What is the business question behind the data ques
- What are the key dimensions, metrics, and segmentations needed?
- What statistical approaches are appropriate?
- What surprises might the data contain?
- What visualizations will make this clearest?
- Edge cases: nulls, outliers, class imbalance, date parsing issues

```
### STEP 2 – PLAN (always use <plan> tags)
```

```
Write a numbered implementation plan with specific steps. Be concrete. Each step sho
```

```
### STEP 3 – CODE (use <code lang="python"> or <code lang="sql"> tags)
```

```
Write production-quality code:
```

- Always handle nulls and data type issues
- Add comments explaining WHY not just what
- Use correct statistical tests for the data type
- CRITICAL: For visualizations, DO NOT use matplotlib.show() or savefig()
Instead, return structured data as JSON for the frontend to render

```
Use this pattern:
```

```
```python
```

```
At end of visualization code, ALWAYS output:
```

```
import json
```

```
chart_data = {
```

```
 "type": "bar", # or "line", "scatter", "pie", "heatmap", "histogram"
```

```
 "title": "Your chart title",
```

```
 "subtitle": "Context or statistical note",
```

```
 "labels": [...],
```

```
 "datasets": [
```

```
 {
```

```

 "label": "Series name",
 "data": [...],
 "color": "#3B82F6" # use from palette: #3B82F6 blue, #EF4444 red, #22C04A green
 }
],
"xLabel": "X axis label",
"yLabel": "Y axis label"
}
print("CHART_DATA:" + json.dumps(chart_data))
...

```

### STEP 4 – INSIGHTS (use <insights> tags)

Generate 3-5 specific, quantified, actionable insights.

Each insight must have:

- A severity: "critical" | "warning" | "positive" | "info" | "model"
- A title: max 10 words, punchy and specific
- Body: 2-4 sentences with real numbers from the data
- Action: What should the business DO about this?

## RESPONSE FORMAT (STRICT JSON OUTPUT)

After your full analysis, output a single JSON block wrapped in <DATAIQ\_RESPONSE> tags

<DATAIQ\_RESPONSE>

```

{
 "thinking": ["thought 1", "thought 2", ...],
 "plan": [
 {"step": 1, "description": "...", "status": "done"},
 ...
],
 "code_blocks": [
 {
 "language": "sql",
 "title": "Segment churn rates",
 "subtitle": "PostgreSQL · aggregate query",
 "code": "SELECT ..."
 },
 {
 "language": "python",
 "title": "Statistical significance",
 "subtitle": "scipy · chi-squared test",
 "code": "import pandas as pd\n..."
 }
],
 "stats": [

```

```

 {"label": "Overall Churn", "value": "26.5%", "delta": "↑ 4.2% vs Q2", "color": "#2E8B57"},
 ...
],
"charts": [
 {
 "type": "bar",
 "title": "Churn Rate by Contract Type",
 "subtitle": " $\chi^2 = 1847.3$, $p < 0.001$ ",
 "labels": ["Month-to-month", "One year", "Two year"],
 "datasets": [{"label": "Churn %", "data": [42.7, 11.3, 2.8], "colors": ["#EF4444", "#2E8B57", "#4682B4"]},
 "xLabel": "Contract Type",
 "yLabel": "Churn Rate (%)"
 }
],
"insights": [
 {
 "severity": "critical",
 "title": "Month-to-Month Contract Churn Crisis",
 "body": "42.7% of month-to-month customers churned in Q3...",
 "action": "Launch contract upgrade campaign with 2-month free incentive"
 }
],
"execution_time": "1.87s",
"data_sources_used": ["customers.csv", "PostgreSQL:transactions"]
}
</DATAIQ_RESPONSE>

```

## ## RULES

- NEVER fabricate numbers. If code produced no result, say so.
- ALWAYS run code in the sandbox and use actual results in your insights
- For ML models, always report train/test split, metrics, and feature importances
- Use correct statistical tests: chi-squared for categoricals, ANOVA for 3+ group means
- When data has issues (nulls, outliers), report them in insights
- Be opinionated. Don't hedge. Tell the user what the data actually means.

"""

## 3.2 Agent Orchestration with Google ADK

```
python
```

```

backend/agent/analyst_agent.py

import asyncio
import json
import re
from typing import AsyncGenerator
from openai import AsyncOpenAI
from .prompts import ANALYST_SYSTEM_PROMPT
from .sandbox import DataIQSandbox
from .data_loader import DataLoader

client = AsyncOpenAI() # uses OPENAI_API_KEY env var

class DataIQAnalystAgent:
 """
 Senior AI Data Analyst Agent.
 Orchestrates: Think → Plan → Code → Sandbox → Parse → Stream
 """

 def __init__(self):
 self.sandbox = DataIQSandbox()
 self.loader = DataLoader()

 async def analyze(
 self,
 prompt: str,
 data_sources: list[dict],
 notebook_context: list[dict] = None,
 use_thinking_model: bool = True
) -> AsyncGenerator[dict, None]:
 """
 Main entry point. Yields SSE events matching prototype shape.
 Each yielded dict becomes one SSE event sent to the frontend.
 """

 # 1. Load data context (schema + sample rows, NOT full data)
 data_context = await self.loader.build_context(data_sources)

 # 2. Build messages
 messages = self._build_messages(prompt, data_context, notebook_context)

 # 3. Stream from OpenAI
 yield {"event": "status", "data": {"message": "AI analyst is thinking...", "

```

```

full_response = ""
async for chunk in self._stream_openai(messages, use_thinking_model):
 full_response += chunk
 # Stream thinking tokens in real-time
 if "<thinking>" in full_response and "</thinking>" not in full_response:
 thinking_text = full_response.split("<thinking>")[-1]
 yield {"event": "thinking_token", "data": {"token": chunk}}

yield {"event": "status", "data": {"message": "Running code in sandbox..."},

4. Extract code blocks and run in sandbox
code_blocks = self._extract_code_blocks(full_response)
sandbox_results = []
for block in code_blocks:
 if block["language"] == "python":
 result = await self.sandbox.run_python(
 block["code"],
 data_sources=data_sources
)
 sandbox_results.append(result)
 yield {"event": "sandbox_result", "data": result}

5. Parse final structured response
structured = self._parse_dataiq_response(full_response, sandbox_results)

6. Yield final complete response (matches prototype shape exactly)
yield {"event": "complete", "data": structured}

def _build_messages(self, prompt, data_context, notebook_context):
 messages = [{"role": "system", "content": ANALYST_SYSTEM_PROMPT}]

 # Add notebook history for context
 if notebook_context:
 for cell in notebook_context[-5:]: # last 5 cells
 messages.append({"role": "user", "content": cell["prompt"]})
 if cell.get("result_summary"):
 messages.append({"role": "assistant", "content": cell["result_su

 # Main user message with data context
 user_message = f"""
Data Sources Available
{json.dumps(data_context, indent=2)}

```

## ## User's Analysis Request

{prompt}

Remember: Return your full analysis including the <DATAIQ\_RESPONSE> JSON block at the  
"""

```
messages.append({"role": "user", "content": user_message})
return messages
```

```
async def _stream_openai(self, messages, use_thinking_model):
```

```
 """
```

```
 Use gpt-4o for fast responses, o1 for deep thinking queries.
```

```
 Detected by prompt complexity / user preference.
```

```
 """
```

```
 model = "o1-preview" if use_thinking_model else "gpt-4o"
```

```
 # o1 doesn't support streaming yet – use gpt-4o with detailed prompt
```

```
 # When o1 supports streaming, swap this out
```

```
 model = "gpt-4o" # TODO: swap to o1 when streaming is GA
```

```
 stream = await client.chat.completions.create(
 model=model,
 messages=messages,
 stream=True,
 temperature=0.1, # Low temp for analytical accuracy
 max_tokens=4000,
)
```

```
 async for chunk in stream:
 delta = chunk.choices[0].delta
 if delta.content:
 yield delta.content
```

```
def _extract_code_blocks(self, response: str) -> list[dict]:
 pattern = r'<code lang="(\w+)">(.*?)</code>'
 blocks = re.findall(pattern, response, re.DOTALL)
 return [{"language": lang, "code": code.strip()} for lang, code in blocks]
```

```
def _parse_dataiq_response(self, response: str, sandbox_results: list) -> dict:
 """Parse the <DATAIQ_RESPONSE> JSON and enrich with real sandbox results."""
 match = re.search(r'<DATAIQ_RESPONSE>(.*?)</DATAIQ_RESPONSE>', response, re.
 if not match:
 return self._fallback_response(response)
```

```
 try:
```

```

 structured = json.loads(match.group(1).strip())
 except json.JSONDecodeError:
 return self._fallback_response(response)

 # Enrich charts with actual sandbox data if sandbox produced CHART_DATA
 for result in sandbox_results:
 if result.get("chart_data"):
 # Merge real chart data from sandbox into structured response
 for i, chart in enumerate(structured.get("charts", [])):
 if i < len(result["chart_data"]):
 chart["datasets"] = result["chart_data"][i].get("datasets",
 chart.get("datasets", []))
 chart["labels"] = result["chart_data"][i].get("labels", chart.get("labels", []))

 return structured

def _fallback_response(self, raw_text: str) -> dict:
 """Graceful fallback if JSON parsing fails."""
 return {
 "thinking": ["Analysis complete"],
 "plan": [{"step": 1, "description": "Analysis performed", "status": "done"}],
 "code_blocks": [],
 "stats": [],
 "charts": [],
 "insights": [{
 "severity": "info",
 "title": "Analysis Complete",
 "body": raw_text[:500],
 "action": "Review the output above"
 }],
 "execution_time": "N/A",
 "data_sources_used": []
 }

```

## 4. SANDBOX INTEGRATION — E2B.DEV (FREE TIER)

### Why E2B?

- **Free tier:** 100 hours/month free, no credit card needed to start
- **URL:** <https://e2b.dev> — sign up, get `E2B_API_KEY`
- **Python SDK:** `pip install e2b`

- Runs real Python in isolated containers with pandas, numpy, scipy, sklearn pre-installed
- Supports file uploads into the sandbox

python

```

backend/agent/sandbox.py

import asyncio
import json
import re
import io
import base64
from e2b import AsyncSandbox
import pandas as pd

class DataIQSandbox:
 """
 Manages E2B sandbox sessions for safe Python code execution.
 Each analysis gets its own sandbox (auto-killed after 5 min).
 """

 async def run_python(self, code: str, data_sources: list[dict]) -> dict:
 """
 Run Python code in E2B sandbox.
 Returns: { stdout, stderr, chart_data, error }
 """

 async with AsyncSandbox(timeout=300) as sandbox: # 5 min timeout

 # 1. Upload data files into sandbox
 for source in data_sources:
 if source["type"] == "file" and source.get("content"):
 await sandbox.files.write(
 f"/data/{source['filename']}",
 source["content"] # bytes
)

 # 2. Inject data loading preamble
 preamble = self._build_preamble(data_sources)
 full_code = preamble + "\n\n" + code

 # 3. Run
 execution = await sandbox.run_code(full_code)

 # 4. Parse output
 stdout = "\n".join([log.line for log in execution.logs.stdout])
 stderr = "\n".join([log.line for log in execution.logs.stderr])

 # 5. Extract CHART_DATA from stdout

```

```

 chart_data = self._extract_chart_data(stdout)

 return {
 "stdout": stdout,
 "stderr": stderr,
 "chart_data": chart_data,
 "error": execution.error.value if execution.error else None,
 "success": execution.error is None
 }

def _build_preamble(self, data_sources: list[dict]) -> str:
 """Auto-load data into sandbox based on source type."""
 lines = [
 "import pandas as pd",
 "import numpy as np",
 "import json",
 "import warnings",
 "warnings.filterwarnings('ignore')",
 "",
]
 for source in data_sources:
 if source["type"] == "file":
 varname = source["filename"].replace(".csv", "").replace(".xlsx", "")
 if source["filename"].endswith(".csv"):
 lines.append(f'{varname} = pd.read_csv("/data/{source["filename"]}')
 elif source["filename"].endswith((".xlsx", ".xls")):
 lines.append(f'{varname} = pd.read_excel("/data/{source["filename"]}')
 lines.append(f'df = {varname} # default df alias')
 elif source["type"] == "postgres":
 # Connection string injected securely
 lines.append(f'import sqlalchemy')
 lines.append(f'engine = sqlalchemy.create_engine("{source["connection_string"]}')
 lines.append(f'# Use: pd.read_sql("SELECT ...", engine)')
 return "\n".join(lines)

def _extract_chart_data(self, stdout: str) -> list[dict]:
 """Extract CHART_DATA: {...} lines from stdout."""
 charts = []
 for line in stdout.split("\n"):
 if line.startswith("CHART_DATA:"):
 try:
 data = json.loads(line[len("CHART_DATA:"):])
 charts.append(data)
 except:

```

```
 pass
 return charts
```

```
Alternative free sandboxes (use any):
1. E2B (recommended): https://e2b.dev – 100 hrs/month free
2. Modal (https://modal.com) – $30/month free credit
3. Self-hosted: Docker container with restricted network
docker run --rm --network none --memory 512m python:3.11-slim python -c "...
4. Pyodide in browser (no backend needed for pure Python, no file I/O)
```

## Self-hosted Docker Sandbox (Zero cost alternative)

```
python
```

```

backend/agent/sandbox_docker.py
Use this if you don't want E2B costs

import asyncio
import tempfile
import os

class DockerSandbox:
 """
 Runs Python in isolated Docker container.
 Requires Docker installed on your server.
 """

 async def run_python(self, code: str, data_sources: list[dict]) -> dict:
 with tempfile.TemporaryDirectory() as tmpdir:
 # Write code file
 code_path = os.path.join(tmpdir, "analysis.py")
 with open(code_path, "w") as f:
 f.write(code)

 # Write data files
 for source in data_sources:
 if source.get("content"):
 with open(os.path.join(tmpdir, source["filename"]), "wb") as f:
 f.write(source["content"])

 cmd = [
 "docker", "run", "--rm",
 "--network", "none", # no internet access
 "--memory", "512m", # memory limit
 "--cpus", "1.0", # CPU limit
 "--pids-limit", "50", # process limit
 "-v", f"{tmpdir}:/workspace",
 "-w", "/workspace",
 "dataiq-sandbox:latest", # your custom image (see Dockerfile below)
 "python", "analysis.py"
]

 proc = await asyncio.create_subprocess_exec(
 *cmd,
 stdout=asyncio.subprocess.PIPE,
 stderr=asyncio.subprocess.PIPE
)

```

```

try:
 stdout, stderr = await asyncio.wait_for(proc.communicate(), timeout=
except asyncio.TimeoutError:
 proc.kill()
 return {"stdout": "", "stderr": "Timeout", "chart_data": [], "error"

stdout_str = stdout.decode("utf-8", errors="replace")
stderr_str = stderr.decode("utf-8", errors="replace")
chart_data = self._extract_chart_data(stdout_str)

return {
 "stdout": stdout_str,
 "stderr": stderr_str,
 "chart_data": chart_data,
 "error": None if proc.returncode == 0 else stderr_str,
 "success": proc.returncode == 0
}

```

dockerfile

```

Dockerfile for self-hosted sandbox
Save as: backend/sandbox/Dockerfile

```

```

FROM python:3.11-slim

```

```

RUN pip install --no-cache-dir \
 pandas==2.2.2 \
 numpy==1.26.4 \
 scipy==1.13.1 \
 scikit-learn==1.5.0 \
 statsmodels==0.14.2 \
 openpyxl==3.1.3 \
 sqlalchemy==2.0.30 \
 psycpg2-binary==2.9.9 \
 matplotlib==3.9.0

```

```

Build: docker build -t dataiq-sandbox:latest ./backend/sandbox/

```

## 5. DATA LOADER — CONNECTING ALL DATA SOURCES

python

```

backend/agent/data_loader.py

import pandas as pd
import io
import json
from sqlalchemy import create_engine, text, inspect
from supabase import create_client
from typing import Optional

class DataLoader:
 """
 Loads schema + sample data from any data source.
 IMPORTANT: Only loads schema + 100 rows for context.
 Full data is loaded inside the sandbox by the agent's code.
 """

 async def build_context(self, data_sources: list[dict]) -> list[dict]:
 """
 Returns a list of data source contexts:
 { name, type, schema, sample_rows, row_count, size_bytes }
 This is what the AI sees when building its plan.
 """
 contexts = []
 for source in data_sources:
 if source["type"] == "file":
 ctx = await self._context_from_file(source)
 elif source["type"] == "postgres":
 ctx = await self._context_from_postgres(source)
 elif source["type"] == "supabase":
 ctx = await self._context_from_supabase(source)
 elif source["type"] == "mcp":
 ctx = await self._context_from_mcp(source)
 else:
 ctx = {"name": source.get("name", "unknown"), "type": source["type"]}
 contexts.append(ctx)
 return contexts

 async def _context_from_file(self, source: dict) -> dict:
 """Handle uploaded CSV/Excel files stored in S3 or local."""
 content = source.get("content") # bytes
 if not content:
 return {"error": "No file content"}

```

```

try:
 if source["filename"].endswith(".csv"):
 df = pd.read_csv(io.BytesIO(content))
 else:
 df = pd.read_excel(io.BytesIO(content))

 schema = {}
 for col in df.columns:
 schema[col] = {
 "dtype": str(df[col].dtype),
 "nulls": int(df[col].isna().sum()),
 "unique": int(df[col].nunique()),
 "sample": df[col].dropna().head(3).tolist()
 }

 return {
 "name": source["filename"],
 "type": "file",
 "schema": schema,
 "row_count": len(df),
 "columns": list(df.columns),
 "sample_rows": df.head(5).to_dict(orient="records"),
 "dtypes_summary": df.dtypes.astype(str).to_dict()
 }
except Exception as e:
 return {"name": source["filename"], "type": "file", "error": str(e)}

async def _context_from_postgres(self, source: dict) -> dict:
 """Get schema from PostgreSQL without ingesting data."""
 try:
 engine = create_engine(source["connection_string"])
 inspector = inspect(engine)

 tables = {}
 for table_name in inspector.get_table_names(schema=source.get("schema",
 columns = inspector.get_columns(table_name)
 with engine.connect() as conn:
 count = conn.execute(text(f'SELECT COUNT(*) FROM "{table_name}"'))
 sample = pd.read_sql(f'SELECT * FROM "{table_name}" LIMIT 5', co

 tables[table_name] = {
 "columns": {col["name"]: str(col["type"]) for col in columns},
 "row_count": count,
 "sample_rows": sample.to_dict(orient="records")

```

```

 }

 return {
 "name": source.get("name", "PostgreSQL"),
 "type": "postgres",
 "tables": tables,
 "connection_available": True # agent will use this connection string
 }
except Exception as e:
 return {"name": "PostgreSQL", "type": "postgres", "error": str(e)}

async def _context_from_mcp(self, source: dict) -> dict:
 """
 For MCP connectors (Google Sheets, Drive, etc.),
 use the MCP client to fetch schema/sample without full ingestion.
 """
 # MCP integration – see Section 7 for full MCP setup
 connector_type = source.get("connector_type")

 if connector_type == "google_sheets":
 return await self._mcp_google_sheets(source)
 elif connector_type == "google_drive":
 return await self._mcp_google_drive(source)
 else:
 return {"name": source.get("name"), "type": "mcp", "error": f"Unknown MCP"}

async def _mcp_google_sheets(self, source: dict) -> dict:
 """Fetch Google Sheets data via MCP."""
 # This calls the MCP server for Google Sheets
 # See Section 7 for full MCP server setup
 from .mcp_client import MCPClient
 client = MCPClient(source["mcp_server_url"])
 data = await client.call_tool("get_sheet_data", {
 "spreadsheet_id": source["spreadsheet_id"],
 "range": source.get("range", "Sheet1!A:Z")
 })
 # Parse and return schema
 df = pd.DataFrame(data["values"][1:], columns=data["values"][0])
 return {
 "name": source.get("name", "Google Sheet"),
 "type": "google_sheets",
 "columns": list(df.columns),
 "row_count": len(df),
 }

```

```
 "sample_rows": df.head(5).to_dict(orient="records")
}
```

---

## 6. FASTAPI ENDPOINTS

python

```
backend/main.py
```

```
from fastapi import FastAPI, UploadFile, File, HTTPException, Depends
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from sse_starlette.sse import EventSourceResponse
from pydantic import BaseModel
import json
import asyncio
from typing import Optional
```

```
from agent.analyst_agent import DataIQAnalystAgent
from db.session import get_db
from models.datasource import DataSource
from models.notebook import Notebook
from storage.file_store import FileStore
```

```
app = FastAPI(title="DataIQ API", version="1.0.0")
```

```
app.add_middleware(
 CORSMiddleware,
 allow_origins=["http://localhost:5173", "https://yourdomain.com"],
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)
```

```
analyst = DataIQAnalystAgent()
file_store = FileStore()
```

```
— REQUEST/RESPONSE MODELS —
```

```
class AnalyzeRequest(BaseModel):
 prompt: str
 data_sources: list[dict] # [{ "id": "...", "type": "file"|"postgres"|"m
 notebook_id: Optional[str] = None
 notebook_context: Optional[list[dict]] = None # previous cells
 use_thinking: bool = True
```

```
class PinInsightRequest(BaseModel):
 notebook_id: str
```

insight: dict

# ——— MAIN ANALYSIS ENDPOINT (SSE STREAMING) ———

```
@app.post("/api/v1/analyze")
```

```
async def analyze(request: AnalyzeRequest):
```

```
 """
```

```
 Main analysis endpoint. Streams SSE events.
```

```
 Frontend connects with EventSource and receives:
```

```
 - { event: "status", data: { message, stage } }
 - { event: "thinking_token", data: { token } }
 - { event: "sandbox_result", data: { stdout, chart_data, error } }
 - { event: "complete", data: { thinking, plan, code_blocks, stats, charts, insight } }
 """
```

```
Resolve data sources (load content from storage for files)
```

```
resolved_sources = await resolve_data_sources(request.data_sources)
```

```
async def event_stream():
```

```
 try:
```

```
 async for event in analyst.analyze(
 prompt=request.prompt,
 data_sources=resolved_sources,
 notebook_context=request.notebook_context,
 use_thinking_model=request.use_thinking
```

```
):

```

```
 yield {
 "event": event["event"],
 "data": json.dumps(event["data"])
 }
```

```
 except Exception as e:
```

```
 yield {
 "event": "error",
 "data": json.dumps({"message": str(e), "type": type(e).__name__})
 }
```

```
 return EventSourceResponse(event_stream())
```

# ——— FILE UPLOAD ———

```
@app.post("/api/v1/upload")
```

```
async def upload_file(
```

```

 file: UploadFile = File(...),
 # user_id: str = Depends(get_current_user) # add auth later
):
 """Upload a file (CSV, Excel, JSON). Returns datasource_id."""
 content = await file.read()
 file_id = await file_store.store(
 filename=file.filename,
 content=content,
 content_type=file.content_type,
 # user_id=user_id
)

 # Quick profile for immediate UI feedback
 import pandas as pd, io
 try:
 df = pd.read_csv(io.BytesIO(content)) if file.filename.endswith(".csv") else
 profile = {
 "rows": len(df),
 "columns": len(df.columns),
 "column_names": list(df.columns),
 "size_bytes": len(content),
 "preview": df.head(5).to_dict(orient="records")
 }
 except:
 profile = {"rows": 0, "columns": 0, "error": "Could not profile file"}

 return {
 "datasource_id": file_id,
 "filename": file.filename,
 "profile": profile,
 "type": "file"
 }

```

# — DATA SOURCES —

---

```

@app.get("/api/v1/datasources")
async def list_datasources():
 """List all data sources for the current user (uploaded files + connectors)."""
 # TODO: filter by user_id when auth is added
 files = await file_store.list()
 connectors = await get_configured_connectors()

 return {

```

```
 "files": files,
 "connectors": connectors
 }
```

```
@app.post("/api/v1/connectors")
async def add_connector(connector: dict):
 """Save a new data connector configuration."""
 # Validate connection before saving
 validated = await validate_connector(connector)
 if not validated["success"]:
 raise HTTPException(400, detail=validated["error"])
 # Save to DB
 # connector_id = await db.connectors.insert(connector)
 return {"connector_id": "generated-id", "status": "connected"}
```

# ——— NOTEBOOKS ———

```
@app.post("/api/v1/notebooks")
async def save_notebook(notebook: dict):
 """Save notebook state (cells, insights, charts)."""
 # notebook_id = await db.notebooks.upsert(notebook)
 return {"notebook_id": "saved-id"}
```

```
@app.get("/api/v1/notebooks/{notebook_id}")
async def get_notebook(notebook_id: str):
 """Load a saved notebook."""
 # return await db.notebooks.get(notebook_id)
 return {}
```

# ——— INSIGHTS ———

```
@app.post("/api/v1/insights/pin")
async def pin_insight(request: PinInsightRequest):
 """Pin an insight to the dashboard."""
 # await db.insights.insert({ notebook_id, insight, pinned_at: now() })
 return {"pinned": True}
```

# ——— HELPERS ———

```

async def resolve_data_sources(sources: list[dict]) -> list[dict]:
 """Attach actual file content to datasource specs."""
 resolved = []
 for source in sources:
 if source["type"] == "file":
 content = await file_store.get(source["id"])
 resolved.append(**source, "content": content)
 else:
 resolved.append(source)
 return resolved

async def get_configured_connectors() -> list[dict]:
 """Return list of configured connectors for the user."""
 # TODO: fetch from DB
 return [
 {"id": "pg-1", "type": "postgres", "name": "Production DB", "status": "connected"},
 {"id": "sb-1", "type": "supabase", "name": "Supabase Analytics", "status": "connected"}
]

async def validate_connector(connector: dict) -> dict:
 """Test connector connection before saving."""
 try:
 if connector["type"] == "postgres":
 from sqlalchemy import create_engine, text
 engine = create_engine(connector["connection_string"])
 with engine.connect() as conn:
 conn.execute(text("SELECT 1"))
 return {"success": True}
 except Exception as e:
 return {"success": False, "error": str(e)}

```

---

## 7. MCP CONNECTOR INTEGRATION

```
python
```

```

backend/agent/mcp_client.py

import asyncio
import json
import aiohttp
from typing import Any

class MCPClient:
 """
 Generic MCP (Model Context Protocol) client.
 Connects to any MCP server via HTTP/SSE.
 """

 def __init__(self, server_url: str, api_key: str = None):
 self.server_url = server_url
 self.headers = {"Authorization": f"Bearer {api_key}" if api_key else {}}

 async def list_tools(self) -> list[dict]:
 async with aiohttp.ClientSession() as session:
 async with session.get(f"{self.server_url}/tools", headers=self.headers) as r:
 data = await r.json()
 return data["tools"]

 async def call_tool(self, tool_name: str, arguments: dict) -> Any:
 async with aiohttp.ClientSession() as session:
 payload = {"name": tool_name, "arguments": arguments}
 async with session.post(
 f"{self.server_url}/call",
 json=payload,
 headers=self.headers
) as r:
 result = await r.json()
 if result.get("error"):
 raise Exception(result["error"])
 return result["content"]

MCP Server configurations
These are the MCP servers you run or connect to for each integration:

MCP_SERVERS = {
 "google_sheets": {
 "url": "https://sheets.googleapis.com/mcp/v1", # or self-hosted
 }
}

```

```

 "description": "Google Sheets read/write access",
 "setup": "Use google-sheets-mcp package from npm: npx @modelcontextprotocol/
 },
 "google_drive": {
 "url": "https://drive.googleapis.com/mcp/v1",
 "description": "Google Drive file access",
 "setup": "npx @modelcontextprotocol/server-google-drive"
 },
 "postgresql": {
 "url": "http://localhost:5433/mcp", # self-hosted
 "description": "PostgreSQL database access",
 "setup": "npx @modelcontextprotocol/server-postgres postgresql://user:pass@h
 },
 "supabase": {
 "url": "https://mcp.supabase.com/v1",
 "description": "Supabase tables and edge functions",
 "setup": "Use Supabase MCP at https://supabase.com/docs/guides/ai/mcp"
 }
}

MCP Server setup commands (run these once per integration):
MCP_SETUP_COMMANDS = """
Google Sheets MCP
npx @modelcontextprotocol/server-google-sheets

PostgreSQL MCP
npx @modelcontextprotocol/server-postgres "postgresql://localhost/mydb"

Filesystem MCP (for local files)
npx @modelcontextprotocol/server-filesystem /path/to/allowed/files

Supabase MCP – see https://supabase.com/blog/supabase-mcp
"""

```

## 8. FRONTEND INTEGRATION — EXACT REACT CODE

### 8.1 Zustand Store (matches prototype state shape exactly)

```
typescript
```

```
// src/store/notebookStore.ts

import { create } from 'zustand'

export interface ThinkingStep {
 text: string
}

export interface PlanStep {
 step: number
 description: string
 status: 'pending' | 'done' | 'running'
}

export interface CodeBlock {
 language: 'python' | 'sql'
 title: string
 subtitle: string
 code: string
}

export interface StatCard {
 label: string
 value: string
 delta: string
 color: 'red' | 'green' | 'blue' | 'amber' | 'neutral'
}

export interface ChartData {
 type: 'bar' | 'line' | 'scatter' | 'pie' | 'histogram' | 'heatmap'
 title: string
 subtitle: string
 labels: string[]
 datasets: Array<{
 label: string
 data: number[]
 colors?: string[]
 color?: string
 }>
 xLabel?: string
 yLabel?: string
}
```

```
export interface Insight {
 severity: 'critical' | 'warning' | 'positive' | 'info' | 'model'
 title: string
 body: string
 action: string
 pinned?: boolean
}
```

```
export interface CellResult {
 thinking: ThinkingStep[]
 plan: PlanStep[]
 code_blocks: CodeBlock[]
 stats: StatCard[]
 charts: ChartData[]
 insights: Insight[]
 execution_time: string
 data_sources_used: string[]
}
```

```
export interface Cell {
 id: string
 prompt: string
 status: 'idle' | 'thinking' | 'running' | 'complete' | 'error'
 result?: CellResult
 streamingThinking?: string
 error?: string
 createdAt: Date
}
```

```
export interface DataSource {
 id: string
 type: 'file' | 'postgres' | 'supabase' | 'mcp'
 name: string
 filename?: string
 status: 'connected' | 'error'
 profile?: {
 rows: number
 columns: number
 column_names: string[]
 }
}
```

```
interface NotebookStore {
 cells: Cell[]
}
```

```

dataSources: DataSource[]
pinnedInsights: Insight[]
notebookTitle: string
addCell: () => string
updateCell: (id: string, updates: Partial<Cell>) => void
runCell: (id: string, prompt: string, selectedSources: DataSource[]) => Promise<void>
pinInsight: (insight: Insight) => void
unpinInsight: (title: string) => void
setDataSources: (sources: DataSource[]) => void
addDataSource: (source: DataSource) => void
}

export const useNotebookStore = create<NotebookStore>((set, get) => ({
 cells: [],
 dataSources: [],
 pinnedInsights: [],
 notebookTitle: 'Untitled Analysis',

 addCell: () => {
 const id = `cell-${Date.now()}`
 set(state => ({
 cells: [...state.cells, {
 id,
 prompt: '',
 status: 'idle',
 createdAt: new Date()
 }]
 })))
 return id
 },

 updateCell: (id, updates) => {
 set(state => ({
 cells: state.cells.map(c => c.id === id ? { ...c, ...updates } : c)
 })))
 },

 runCell: async (id, prompt, selectedSources) => {
 const { updateCell } = get()

 updateCell(id, { status: 'thinking', prompt, streamingThinking: '' })

 try {
 const response = await fetch('/api/v1/analyze', {

```

```

method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({
 prompt,
 data_sources: selectedSources.map(s => ({
 id: s.id,
 type: s.type,
 name: s.name,
 filename: s.filename
 })),
 use_thinking: true
})
})

if (!response.ok) throw new Error(`HTTP ${response.status}`)

// Parse SSE stream
const reader = response.body!.getReader()
const decoder = new TextDecoder()
let buffer = ''

while (true) {
 const { done, value } = await reader.read()
 if (done) break

 buffer += decoder.decode(value, { stream: true })
 const lines = buffer.split('\n')
 buffer = lines.pop() || ''

 for (const line of lines) {
 if (line.startsWith('data: ')) {
 try {
 const payload = JSON.parse(line.slice(6))
 const eventLine = lines.find(l => l.startsWith('event: '))
 const eventType = eventLine?.slice(7) || 'message'

 if (eventType === 'status') {
 updateCell(id, { status: payload.stage === 'thinking' ? 'thinking' :

 } else if (eventType === 'thinking_token') {
 set(state => ({
 cells: state.cells.map(c =>
 c.id === id
 ? { ...c, streamingThinking: (c.streamingThinking || '') + pay

```

```

 : c
)
)))

 } else if (eventType === 'complete') {
 // payload matches CellResult shape exactly
 updateCell(id, {
 status: 'complete',
 result: payload,
 streamingThinking: undefined
 })

 } else if (eventType === 'error') {
 updateCell(id, { status: 'error', error: payload.message })
 }
 } catch (e) {
 // ignore malformed SSE lines
 }
 }
}

} catch (error: any) {
 updateCell(id, { status: 'error', error: error.message })
}
},

pinInsight: (insight) => {
 set(state => ({
 pinnedInsights: [...state.pinnedInsights.filter(i => i.title !== insight.title)
]))
},

unpinInsight: (title) => {
 set(state => ({
 pinnedInsights: state.pinnedInsights.filter(i => i.title !== title)
]))
},

setDataSources: (sources) => set({ dataSources: sources }),
addDataSource: (source) => set(state => ({ dataSources: [...state.dataSources, source]
}))

```

## 8.2 API Service Layer

typescript

```

// src/services/api.ts

const BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:8000'

export const api = {
 async uploadFile(file: File, onProgress?: (pct: number) => void) {
 const formData = new FormData()
 formData.append('file', file)

 return new Promise<{ datasource_id: string; profile: any }>((resolve, reject) => {
 const xhr = new XMLHttpRequest()
 xhr.open('POST', `${BASE_URL}/api/v1/upload`)

 xhr.upload.onprogress = (e) => {
 if (onProgress && e.lengthComputable) {
 onProgress(Math.round((e.loaded / e.total) * 100))
 }
 }

 xhr.onload = () => {
 if (xhr.status === 200) resolve(JSON.parse(xhr.responseText))
 else reject(new Error(xhr.responseText))
 }
 xhr.onerror = () => reject(new Error('Upload failed'))
 xhr.send(formData)
 })
 },

 async getDataSources() {
 const res = await fetch(`${BASE_URL}/api/v1/datasources`)
 return res.json()
 },

 async addConnector(connector: any) {
 const res = await fetch(`${BASE_URL}/api/v1/connectors`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(connector)
 })
 if (!res.ok) throw new Error(await res.text())
 return res.json()
 },
}

```

```

async saveNotebook(notebook: any) {
 const res = await fetch(`${BASE_URL}/api/v1/notebooks`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify(notebook)
 })
 return res.json()
},

async pinInsight(notebookId: string, insight: any) {
 const res = await fetch(`${BASE_URL}/api/v1/insights/pin`, {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ notebook_id: notebookId, insight })
 })
 return res.json()
}
}

```

### 8.3 Chart Renderer Component (Matches Prototype Exactly)

tsx

```

// src/components/ChartRenderer.tsx
// This is the ONLY chart rendering component. Takes structured data from API.

import React from 'react'
import {
 BarChart, Bar, LineChart, Line, ScatterChart, Scatter,
 PieChart, Pie, Cell, XAxis, YAxis, CartesianGrid,
 Tooltip, Legend, ResponsiveContainer
} from 'recharts'
import type { ChartData } from '../store/notebookStore'

const COLOR_PALETTE = [
 '#3B82F6', '#EF4444', '#22C55E', '#F59E0B',
 '#A78BFA', '#2DD4BF', '#EC4899', '#F97316'
]

interface Props {
 chart: ChartData
}

export const ChartRenderer: React.FC<Props> = ({ chart }) => {
 // Transform API data format into Recharts format
 const rechartsData = chart.labels.map((label, i) => {
 const point: any = { name: label }
 chart.datasets.forEach(ds => {
 point[ds.label] = ds.data[i]
 })
 return point
 })

 const commonProps = {
 data: rechartsData,
 margin: { top: 10, right: 20, left: 10, bottom: 10 }
 }

 const renderChart = () => {
 switch (chart.type) {
 case 'bar':
 return (
 <BarChart {...commonProps}>
 <CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.05)" />
 <XAxis dataKey="name" tick={{ fill: '#94A3B8', fontSize: 11 }} />
 <YAxis tick={{ fill: '#64748B', fontSize: 10 }} />
 </BarChart>
)
 }
 }
}

```

```

 tickFormatter={v => chart.yLabel?.includes('%') ? `${v}%` : v} />
<Tooltip
 contentStyle={{ background: '#1E293B', border: '1px solid rgba(255,255,255,0.05)' }}
 labelStyle={{ color: '#94A3B8' }}
/>
{chart.datasets.map((ds, i) => {
 const colors = ds.colors || [COLOR_PALETTE[i % COLOR_PALETTE.length]]
 return (
 <Bar key={ds.label} dataKey={ds.label} radius={[4, 4, 0, 0]}>
 {rechartsData.map((_, j) => (
 <Cell key={j} fill={colors[j] || colors[0]} fillOpacity={0.85} />
))}
 </Bar>
)
})}
</BarChart>
)

case 'line':
 return (
 <LineChart {...commonProps}>
 <CartesianGrid strokeDasharray="3 3" stroke="rgba(255,255,255,0.05)" />
 <XAxis dataKey="name" tick={{ fill: '#94A3B8', fontSize: 11 }} />
 <YAxis tick={{ fill: '#64748B', fontSize: 10 }} />
 <Tooltip
 contentStyle={{ background: '#1E293B', border: '1px solid rgba(255,255,255,0.05)' }}
 />
 {chart.datasets.map((ds, i) => (
 <Line
 key={ds.label}
 type="monotone"
 dataKey={ds.label}
 stroke={ds.color || COLOR_PALETTE[i]}
 strokeWidth={2.5}
 dot={{ fill: ds.color || COLOR_PALETTE[i], r: 4 }}
 activeDot={{ r: 6 }}
 />
))}
 </LineChart>
)

case 'pie':
 const pieData = chart.labels.map((label, i) => ({
 name: label,

```

```

 value: chart.datasets[0]?.data[i] || 0
 })))
 return (
 <PieChart>
 <Pie data={pieData} cx="50%" cy="50%" outerRadius={90}
 label={({ name, percent }) => ` ${name} ${(percent * 100).toFixed(1)}%`
 labellLine={({ stroke: '#64748B' })}>
 {pieData.map((_, i) => (
 <Cell key={i} fill={COLOR_PALETTE[i % COLOR_PALETTE.length]} />
))}
 </Pie>
 <Tooltip contentStyle={{ background: '#1E293B', border: '1px solid rgba(
 </PieChart>
)

 default:
 return <div style={{ color: '#64748B', fontSize: 12, padding: 20 }}>Chart ty
 }
 }

 return (
 <div style={{
 background: 'var(--gray-800, #1E293B)',
 borderRadius: 10,
 padding: '16px',
 border: '1px solid rgba(255,255,255,0.06)',
 marginBottom: 8
 }}>
 <div style={{ fontSize: 13, fontWeight: 600, color: '#E2E8F0', marginBottom: 2
 <div style={{ fontSize: 11, color: '#64748B', marginBottom: 14 }}>{chart.subti
 <ResponsiveContainer width="100%" height={220}>
 {renderChart()}
 </ResponsiveContainer>
 </div>
)
}

```

## 8.4 Streaming Cell Output Component

tsx

```

// src/components/CellOutput.tsx
// Replaces mock output – renders real streaming API response
// IMPORTANT: All section IDs and class names must match the prototype

import React, { useState } from 'react'
import { ChartRenderer } from './ChartRenderer'
import type { Cell } from '../store/notebookStore'
import { useNotebookStore } from '../store/notebookStore'

interface Props {
 cell: Cell
}

const severityConfig = {
 critical: { bg: 'ic-red', icon: '🚨' },
 warning: { bg: 'ic-amber', icon: '⚠️' },
 positive: { bg: 'ic-green', icon: '✅' },
 info: { bg: 'ic-blue', icon: '💡' },
 model: { bg: 'ic-purple', icon: '🤖' }
}

export const CellOutput: React.FC<Props> = ({ cell }) => {
 const [thinkingOpen, setThinkingOpen] = useState(false)
 const { pinInsight } = useNotebookStore()

 // LOADING STATE – matches prototype loading animation exactly
 if (cell.status === 'thinking' || cell.status === 'running') {
 return (
 <div className="output-cell animate-in">
 <div className="output-wrapper">
 {cell.streamingThinking && (
 <div className="thinking-block">
 <div className="tb-header">
 Reasoning
 thinking...
 </div>
 <div className="tb-body open" style={{ display: 'block', paddingTop: 10px }}>
 <div style={{ fontSize: 12, color: '#94A3B8', lineHeight: 1.7, whiteSpace: 'pre-wrap' }}>
 {cell.streamingThinking}

 </div>
 </div>
 </div>
)}
 </div>
 </div>
)
 }

```

```

 })
 <div className="progress-bar-wrap">
 <div className="progress-bar-fill" style={{ width: cell.status === 'thin
 </div>
 <div className="typing-indicator">
 <div className="typing-dot" /><div className="typing-dot" /><div classNa
 </div>
 </div>
</div>
)
}

if (cell.status === 'error') {
 return (
 <div className="output-cell animate-in">
 <div className="output-wrapper">
 <div className="insight-card ic-red">
 ✖
 <div className="ic-body">
 <div className="ic-title">Analysis Error</div>
 <div className="ic-text">{cell.error}</div>
 </div>
 </div>
 </div>
 </div>
)
}

if (!cell.result) return null
const { thinking, plan, code_blocks, stats, charts, insights, execution_time } = c

return (
 <div className="output-cell animate-in">
 <div className="output-wrapper">

 {/* THINKING BLOCK */}
 {thinking?.length > 0 && (
 <div className="thinking-block">
 <div className="tb-header" onClick={() => setThinkingOpen(!thinkingOpen)}>
 <svg className="tb-icon" viewBox="0 0 16 16" fill="none" stroke="#A78B
 <path d="M8 2a5 5 0 0 1 5 5c0 1.5-.7 2.9-1.8 3.8L11 13H5l-.2-2.2A5 5
 <path d="M6 14h4"/>
 </svg>
 Reasoning

```

```

 {thinking.length} thoughts
 ▼
 </div>
 {thinkingOpen && (
 <div className="tb-body open">
 {thinking.map((t, i) => (
 <div key={i} className="thinking-step">
 <div className="ts-dot" />
 <div className="ts-text">{typeof t === 'string' ? t : t.text}</div>
 </div>
))}
 </div>
)}
</div>
)}

{/* PLAN BLOCK */}
{plan?.length > 0 && (
 <div className="plan-block">
 <div className="pb-title">Implementation Plan</div>
 <div className="pb-steps">
 {plan.map((step, i) => (
 <div key={i} className={`pb-step ${step.status === 'done' ? 'done' : ''}`}>
 <div className="pb-step-num">{step.status === 'done' ? '✓' : step.status}</div>
 {step.description}
 </div>
))}
 </div>
 </div>
)}

{/* CODE BLOCKS */}
{code_blocks?.map((block, i) => (
 <div key={i} className="code-block">
 <div className="cb-header">
 <span className={`pc-type-badge badge-${block.language}`} style={{ font
 {block.language.toUpperCase()}

 {block.subtitle}
 <button className="cb-copy" onClick={() => navigator.clipboard.writeText
 </div>
 <div className="cb-code">
 {/* Use react-syntax-highlighter for real syntax highlighting */}
 <pre style={{ margin: 0, fontFamily: 'JetBrains Mono, monospace', font

```

```

 {block.code}
 </pre>
 </div>
 </div>
))}

{/* STAT CARDS */}
{stats?.length > 0 && (
 <div className="stat-row">
 {stats.map((stat, i) => (
 <div key={i} className="stat-card">
 <div className="sc-label">{stat.label}</div>
 <div className="sc-value" style={{
 color: stat.color === 'red' ? '#FCA5A5' : stat.color === 'green' ?
 }}>{stat.value}</div>
 <div className={`sc-delta ${stat.delta?.includes('↑') ? 'delta-up' :
 </div>
)}}
 </div>
)}
)}

{/* CHARTS – rendered by ChartRenderer */}
{charts?.map((chart, i) => (
 <ChartRenderer key={i} chart={chart} />
))}

{/* INSIGHTS */}
{insights?.length > 0 && (
 <div className="insights-block">
 <div style={{ fontSize: 11, fontWeight: 600, color: '#64748B', textTrans
 🧠 AI Insights
 </div>
 {insights.map((insight, i) => {
 const config = severityConfig[insight.severity] || severityConfig.info
 return (
 <div key={i} className={`insight-card ${config.bg}`}>
 {config.icon}
 <div className="ic-body">
 <div className="ic-title">{insight.title}</div>
 <div className="ic-text">{insight.body}</div>
 {insight.action && (
 <div style={{ marginTop: 6, fontSize: 11, color: '#64748B' }}>
 🗨 {insight.action}
 </div>
)}
 </div>
 </div>
)
)}
)}

```

```

 })
 </div>
 <button
 className={`ic-pin-btn ${insight.pinned ? 'pinned' : ''}`}
 onClick={() => pinInsight({ ...insight, pinned: true })}
 >
 📌 {insight.pinned ? 'Pinned' : 'Pin'}
 </button>
 </div>
)
 })}
</div>
)}

</div>
<div className="exec-status">
 <div className="exec-dot success" />
 Analysis complete · {cell.result.data_sources_used?.join(' + ')}
 {execution_time}
</div>
</div>
)
}

```

## 9. ENVIRONMENT VARIABLES

```
bash
```

```
backend/.env

OpenAI
OPENAI_API_KEY=sk-...

E2B Sandbox (get free key at https://e2b.dev)
E2B_API_KEY=e2b...

Google ADK (for orchestration)
GOOGLE_ADK_PROJECT_ID=your-project-id
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json

Database (for notebooks/users/connectors)
DATABASE_URL=postgresql://user:pass@localhost:5432/dataiq

File Storage (use local for dev, S3 for prod)
STORAGE_BACKEND=local # or "s3"
LOCAL_UPLOAD_DIR=./uploads
AWS_BUCKET_NAME=dataiq-uploads
AWS_REGION=us-east-1

Supabase (if using as connector)
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_KEY=eyJhbGc...

Frontend URL for CORS
FRONTEND_URL=http://localhost:5173
```

```
bash

frontend/.env

VITE_API_URL=http://localhost:8000
VITE_USE MOCK=false # set true to use mock data (prototype mode)
```

---

## 10. FILE STORE (UPLOAD HANDLER)

```
python
```

```

backend/storage/file_store.py

import uuid
import os
import aiofiles
from pathlib import Path

class FileStore:
 """
 Simple file store. Local for dev, S3 for prod.
 Switch via STORAGE_BACKEND env var.
 """

 def __init__(self):
 self.backend = os.getenv("STORAGE_BACKEND", "local")
 self.local_dir = Path(os.getenv("LOCAL_UPLOAD_DIR", "./uploads"))
 self.local_dir.mkdir(exist_ok=True)

 async def store(self, filename: str, content: bytes, content_type: str, user_id:
 file_id = str(uuid.uuid4())
 safe_name = f"{file_id}_{filename}"

 if self.backend == "local":
 path = self.local_dir / safe_name
 async with aiofiles.open(path, "wb") as f:
 await f.write(content)
 elif self.backend == "s3":
 import boto3
 s3 = boto3.client("s3")
 s3.put_object(
 Bucket=os.getenv("AWS_BUCKET_NAME"),
 Key=f"uploads/{safe_name}",
 Body=content,
 ContentType=content_type
)

 # Save metadata to DB
 # await db.files.insert({ id: file_id, filename, safe_name, user_id, size: 1

 return file_id

 async def get(self, file_id: str) -> bytes:
 """Retrieve file content by ID."""

```

```

In real implementation: fetch from DB to get safe_name, then load
For now: scan local dir
for f in self.local_dir.iterdir():
 if f.name.startswith(file_id):
 async with aiofiles.open(f, "rb") as fp:
 return await fp.read()
raise FileNotFoundError(f"File {file_id} not found")

async def list(self) -> list[dict]:
 """List all files (filtered by user in real implementation)."""
 files = []
 for f in self.local_dir.iterdir():
 if f.is_file():
 parts = f.name.split("_", 1)
 files.append({
 "id": parts[0],
 "filename": parts[1] if len(parts) > 1 else f.name,
 "size": f.stat().st_size,
 "type": "file"
 })
 return files

```

## 11. PROJECT DIRECTORY STRUCTURE

```

dataiq/
├─ backend/
│ ├── main.py # FastAPI app entry point
│ ├── requirements.txt
│ ├── .env
│ └─ agent/
│ ├── __init__.py
│ ├── analyst_agent.py # Main agent orchestration
│ ├── prompts.py # System prompts
│ ├── sandbox.py # E2B sandbox runner
│ ├── sandbox_docker.py # Docker alternative
│ ├── data_loader.py # Multi-source data context builder
│ └─ mcp_client.py # MCP connector client
│ └─ storage/
│ └─ file_store.py # File upload handler
└─ models/ # SQLAlchemy models

```

```
| | └─ notebook.py
| | └─ datasource.py
| | └─ insight.py
| └─ sandbox/
| └─ Dockerfile # Self-hosted sandbox image
|
└─ frontend/
 └─ src/
 └─ store/
 └─ notebookStore.ts # Zustand store
 └─ services/
 └─ api.ts # API layer
 └─ components/
 └─ CellOutput.tsx # Streaming output renderer
 └─ ChartRenderer.tsx # Recharts wrapper
 └─ DataSourcePicker.tsx
 └─ FileUpload.tsx
 └─ ...existing components
 └─ package.json
 └─ .env
```

---

## 12. STARTUP COMMANDS

```
bash

Backend
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000

Frontend
cd frontend
npm install
npm run dev

Build sandbox Docker image (optional, if not using E2B)
docker build -t dataiq-sandbox:latest ./backend/sandbox/

Run MCP servers (as needed)
npx @modelcontextprotocol/server-postgres "postgresql://localhost/mydb" &
npx @modelcontextprotocol/server-google-sheets &
```

---

## 13. IMPLEMENTATION ORDER FOR YOUR CODING AGENT

Follow this exact order to avoid breaking anything:

1. **Add packages** to `requirements.txt` and `package.json` — install, verify no conflicts
2. **Create** `backend/agent/prompts.py` — just the system prompt, no wiring yet
3. **Create** `backend/storage/file_store.py` — test file upload endpoint standalone
4. **Create** `backend/agent/sandbox.py` — test with a simple `print("hello")` script
5. **Create** `backend/agent/data_loader.py` — test with an uploaded CSV
6. **Create** `backend/agent/analyst_agent.py` — wire prompts + sandbox + loader
7. **Add FastAPI endpoints** in `backend/main.py` — `/analyze` last (depends on above)
8. **Create** `src/store/notebookStore.ts` — replace any existing mock store
9. **Create** `src/services/api.ts` — test each endpoint from browser console
10. **Create** `src/components/ChartRenderer.tsx` — test with hardcoded chart data first
11. **Create** `src/components/CellOutput.tsx` — swap in for mock output component
12. **Wire** `runCell` in the existing `PromptCell` component to use the store
13. **Test end-to-end** with a real CSV upload and a simple question
14. **Set** `VITE_USE MOCK=false` — prototype experience should be identical, now real

---

## 14. RESOURCE LINKS

Resource	URL	Cost
E2B Sandbox	<a href="https://e2b.dev">https://e2b.dev</a>	Free: 100 hrs/month
OpenAI API	<a href="https://platform.openai.com">https://platform.openai.com</a>	Pay-per-use
Google ADK	<a href="https://google.github.io/adk-docs">https://google.github.io/adk-docs</a>	Free
MCP Python SDK	<a href="https://github.com/modelcontextprotocol/python-sdk">https://github.com/modelcontextprotocol/python-sdk</a>	Free/OSS
MCP Servers (npm)	<a href="https://github.com/modelcontextprotocol/servers">https://github.com/modelcontextprotocol/servers</a>	Free/OSS
Supabase MCP	<a href="https://supabase.com/docs/guides/ai/mcp">https://supabase.com/docs/guides/ai/mcp</a>	Free tier

Resource	URL	Cost
PostgreSQL MCP	<code>npx @modelcontextprotocol/server-postgres</code>	Free/OSS
Google Sheets MCP	<code>npx @modelcontextprotocol/server-google-sheets</code>	Free/OSS
Modal (sandbox alt)	<a href="https://modal.com">https://modal.com</a>	\$30/month free credit
Recharts	<a href="https://recharts.org">https://recharts.org</a>	Free/OSS
Zustand	<a href="https://zustand-demo.pmnd.rs">https://zustand-demo.pmnd.rs</a>	Free/OSS

*This guide is self-contained. Give it to your coding agent as a single document. Every section depends on the ones before it. Follow the implementation order in Section 13.*